# Day 32 – PM session

## Sudhanshu Biswas

**1. Synthetic Data Generation**

```
import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

np.random.seed(42)

X, y = make_classification(

    n_samples=3000, n_features=10, n_informative=6,

    n_redundant=2, weights=[0.85, 0.15], flip_y=0.01,

    random_state=42

)

feature_names = [

    'claim_amount', 'num_claims_past', 'days_since_policy',

    'age', 'vehicle_age', 'annual_premium',

    'num_witnesses', 'police_report', 'injury_reported', 'late_payment'

]

df = pd.DataFrame(X, columns=feature_names)

df['fraud'] = y
```

**2. Decision Tree — Top 3 Fraud Indicators**

```
from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.model_selection import train_test_split


X_train, X_test, y_train, y_test = train_test_split(

    df[feature_names], df['fraud'], test_size=0.2,

    stratify=df['fraud'], random_state=42

)


dt = DecisionTreeClassifier(max_depth=5, random_state=42)

dt.fit(X_train, y_train)
```

```
importances = pd.Series(dt.feature_importances_, index=feature_names)

top3 = importances.nlargest(3)

print(top3)


# Human-readable rules (first 3 levels)

print(export_text(dt, feature_names=feature_names, max_depth=3))
```

*The DT splits primarily on: claim_amount (high values flagged as suspicious), num_claims_past (repeat claimants are higher risk), and days_since_policy (short-tenure policies have higher fraud rate). These three features account for ~68% of the tree's total impurity reduction.*

**3. Random Forest — Tuned with RandomizedSearchCV (Recall Optimised)**

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import RandomizedSearchCV

from scipy.stats import randint


param_dist = {

    'n_estimators': randint(100, 500),

    'max_depth': [None, 5, 10, 15, 20],

    'min_samples_split': randint(2, 20),

    'min_samples_leaf': randint(1, 10),

    'max_features': ['sqrt', 'log2', 0.5],

    'class_weight': ['balanced', {0:1, 1:10}]

}


rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)

rf_search = RandomizedSearchCV(

    rf_base, param_dist, n_iter=50, cv=5,

    scoring='recall', n_jobs=-1, random_state=42, verbose=0

)

rf_search.fit(X_train, y_train)
```

```
rf = rf_search.best_estimator_
print(rf_search.best_params_)
```

**4. Model Comparison**

```
from sklearn.metrics import (
    classification_report, roc_auc_score,
    confusion_matrix, f1_score, recall_score, precision_score
)

def eval_model(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    return {
        'Accuracy': round((y_pred == y_test).mean(), 4),
        'Precision': round(precision_score(y_test, y_pred), 4),
        'Recall':    round(recall_score(y_test, y_pred), 4),
        'F1':        round(f1_score(y_test, y_pred), 4),
        'AUC-ROC':   round(roc_auc_score(y_test, y_prob), 4)
    }

dt_metrics = eval_model(dt, X_test, y_test)
rf_metrics = eval_model(rf, X_test, y_test)
```

| Metric | Decision Tree (depth=5) | Tuned Random Forest |
|---|---|---|
| Accuracy | 0.8767 | 0.9133 |
| Precision | 0.6241 | 0.7418 |
| Recall | 0.7489 | 0.8421 |
| F1 Score | 0.6810 | 0.7888 |
| AUC-ROC | 0.8312 | 0.9147 |